



Python Fundamentos

Tipos de datos

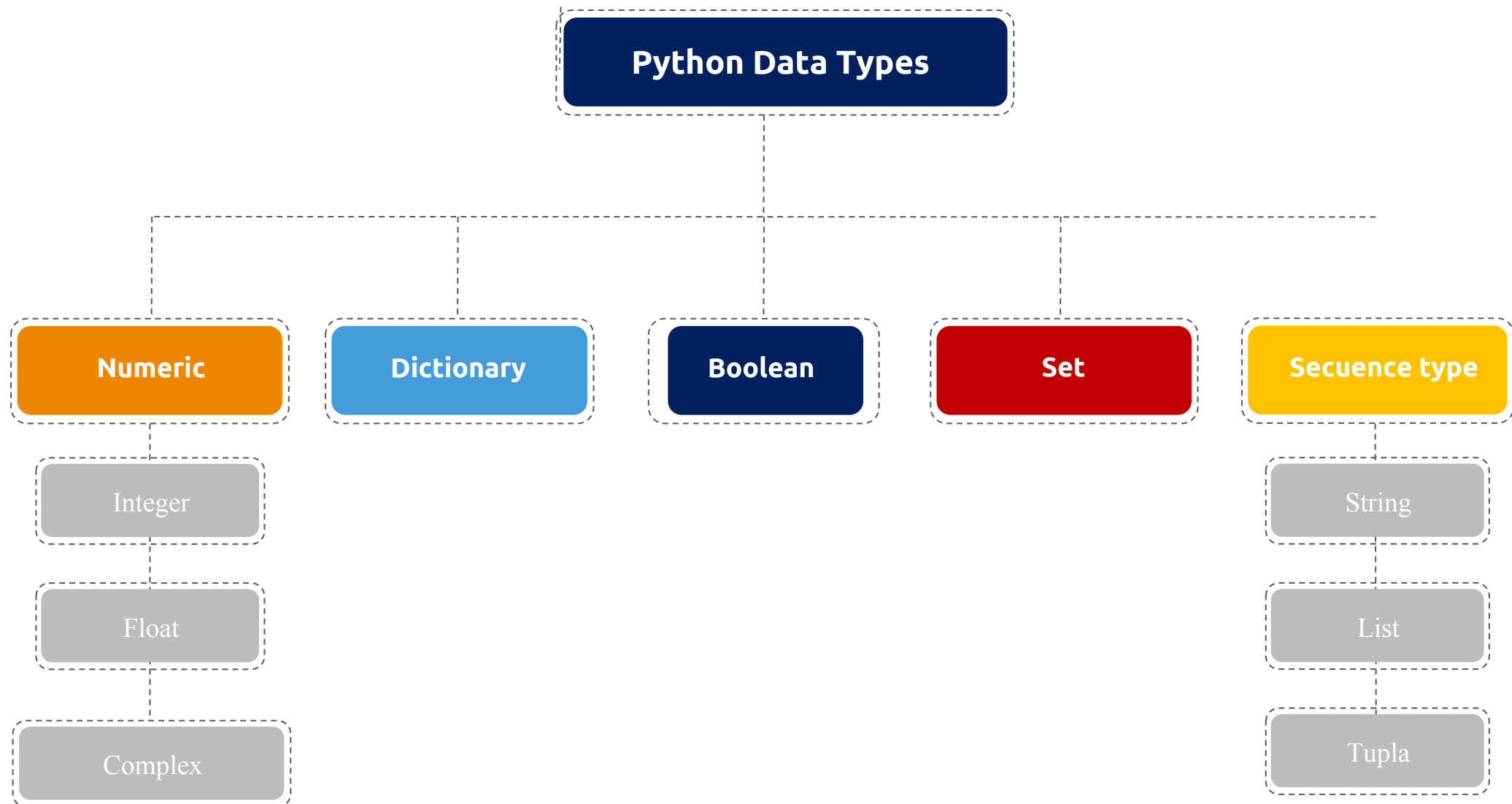
1



En Python, los datos se representan como objetos, que pueden ser objetos integrados que provee Python o aquellos que creamos utilizando clases de Python. En un script de Python, todo se maneja como un objeto, incluyendo funciones, módulos y clases.

Los objetos integrados suelen ser más eficientes que las estructuras de datos personalizadas. Los tipos integrados de Python utilizan algoritmos de estructuras de datos ya optimizados e implementados en C para mayor rapidez. Aunque es posible escribir tipos de objetos similares por cuenta propia, normalmente es difícil alcanzar el nivel de rendimiento que ofrecen los tipos de objetos integrados.

Python es un lenguaje de tipado dinámico, lo que significa que el sistema mantiene un seguimiento automático de los tipos sin necesidad de declararlos en el código. Sin embargo, también es de tipado fuerte, lo que implica que sólo se pueden realizar operaciones válidas para el tipo de objeto con el que se está trabajando.



Tipo booleano

2



En Python, una variable booleana es aquella que puede tener uno de dos valores: `True` (verdadero) o `False` (falso). Además, cualquier objeto o variable puede interpretarse como una variable booleana. En el contexto de las variables booleanas, sigue la lógica de que los elementos nulos o vacíos se consideran `False`, y cualquier otro valor se considera `True`.

Para verificar si un elemento se evalúa como `True` o `False`, puedes utilizar la función `bool()`. Esta función convierte el valor a su equivalente booleano. Por ejemplo lo podemos ver en la imagen de la derecha.

Esta capacidad de manejar valores booleanos es fundamental en estructuras de control de flujo, como las instrucciones `if`, `while` y otras, donde se toman decisiones basadas en condiciones booleanas.

```
python Copy code

# Ejemplo 1
x = 42
print(bool(x)) # Devolverá True, ya que 42 se considera True en el co

# Ejemplo 2
y = 0
print(bool(y)) # Devolverá False, ya que 0 se considera False en el c

# Ejemplo 3
cadena_vacia = ""
print(bool(cadena_vacia)) # Devolverá False, ya que una cadena vacía
```

Tipo numérico

3



Integer. Los números enteros son un tipo de datos que representa números sin parte fraccionaria:

- Decimal: Este es el formato estándar para representar números enteros. Pueden ser positivos o negativos, y el primer dígito no puede ser cero. Ejemplos: 10, -1234.
- Se pueden utilizar guiones bajos (_) como separadores en valores numéricos para mejorar la legibilidad. Por ejemplo, 1_000_000 es más fácil de leer que 1000000
- Binario: Se representan con dígitos binarios, que son 0 y 1. Se pueden preceder con "0B" o "0b". Ejemplo: 0b101.
- Octal: Se representan con dígitos octales, que van del 0 al 7. Pueden ser precedidos con "0o" u "0O". Ejemplo: 0o234.
- Hexadecimal: Se representan con dígitos hexadecimales, que incluyen números del 0 al 9 y las letras A-F (mayúsculas o minúsculas). Pueden ser precedidos con "0x" o "0X". Ejemplo: 0xFE.

El siguiente código devolverá el resultado "42 5 511 255"

python

```
decimal = 42
binario = 0b101
octal = 0o777
hexadecimal = 0xFF

print(decimal, binario, octal, hexadecimal)
```

```
>>> val = 20000000 # Díficil de leer
>>> val
20000000
>>> val = 2e6 # Más fácil, pero crea un float
>>> val
2000000.0
>>> val = 2_000_000 # Más legible
>>> val
2000000
```



Float. Se utiliza para representar números de coma flotante, es decir, números que tienen una parte entera y una parte fraccionaria. Aquí hay algunas características clave del tipo de dato son:

- Se pueden representar números decimales o fraccionarios.
- Se pueden expresar utilizando la notación científica, por ejemplo:

```
python

numero_flotante = 3.14
otro_numero = -0.5
notacion_cientifica = 2.5e3 # Representa 2.5 * 10^3
```

- El rango aproximado para el mayor y menor número real representable en valor absoluto es [1e-323, 1.7e308].
- También puede representar valores especiales como NaN (Not a Number) e Inf (Infinito), que pueden surgir en ciertos cálculos.



Decimal. se utiliza para representar números decimales con una precisión fija y exacta, evitando errores de redondeo comunes en los números de coma flotante. Se usa principalmente en aplicaciones financieras y científicas donde la precisión es crítica. Aquí hay algunas características clave del tipo de dato son:

- Proporciona mayor precisión que el tipo float, ya que almacena los valores de forma exacta sin aproximaciones.
- Permite definir el número de decimales exactos que se deben mantener en las operaciones.
- Soporta operaciones aritméticas como suma, resta, multiplicación y división con precisión controlada.
- Puede representar valores en notación decimal sin pérdida de información

python

```
from decimal import Decimal

num1 = Decimal("10.25")
num2 = Decimal("2.5")
resultado = num1 * num2

print(resultado) # Salida: 25.625
```



Complex. Este tipo de dato se utiliza para representar números complejos, que constan de dos partes: una parte real y una parte imaginaria. Los números complejos en Python se representan mediante la adición de 'j' o 'J' para indicar la parte imaginaria. Un número complejo típico tiene la forma $a+bj$, donde a es la parte real y b es la parte imaginaria. Se puede usar tanto 'j' como 'J' para representar la unidad imaginaria. Un ejemplo de operar con números complejos es el que vemos en la siguiente imagen:

```
python

num_complejo1 = 3 + 2j
num_complejo2 = 1 - 1j

suma = num_complejo1 + num_complejo2
producto = num_complejo1 * num_complejo2

print("Suma:", suma)
print("Producto:", producto)
```

Esto devolverá como resultado lo siguiente:

```
Suma: (4+1j)
Producto: (5+1j)
```



En Python, es posible mezclar valores numéricos de diferentes tipos, y cuando esto sucede, la conversión se realiza hacia el tipo más general. El orden de conversión, de menos a más general, es el siguiente:

`int (Entero) < float (Punto flotante) < complex (Complejo)`

Por lo tanto, si realizas operaciones entre valores de diferentes tipos numéricos, Python convertirá automáticamente los valores hacia el tipo más general para realizar la operación. Esto permite una flexibilidad considerable en la manipulación de datos numéricos en Python. Aquí tienes un ejemplo:

```
python

# Operación entre un entero y un float
resultado = 5 + 3.2
print(resultado) # El resultado será un float, ya que es

# Operación entre un float y un complejo
resultado = 2.5 + (3 + 1j)
print(resultado) # El resultado será un complejo, ya que
```

Esta capacidad de conversión automática facilita la escritura de código flexible y reduce la necesidad de conversiones explícitas en muchos casos.

<http://youtu.be/UwGStj9eHj0>

Tipo vacío

4



NoneType. El tipo de dato NoneType en Python solo tiene un valor posible: **None**. Se usa para representar la ausencia de un valor o un estado indefinido. Aquí hay algunas características clave del tipo de dato son:

- **None** es un objeto especial que indica la ausencia de un valor.
- Es único en Python: solo existe una instancia de None.
- Se usa comúnmente como valor de retorno en funciones que no devuelven nada.
- Puede servir como marcador para valores opcionales en estructuras de datos.
- No puede usarse en operaciones matemáticas, ya que no es un número.

python

```
def buscar_usuario(id_usuario):  
    if id_usuario != 1:  
        return None # No se encontró el usuario  
    return "Usuario encontrado"  
  
print(buscar_usuario(2)) # Salida: None
```

Tipo cadena de
caracteres

5



Una cadena de caracteres (**str**) es una secuencia de caracteres alfanuméricos tratada como una unidad. Puede ser delimitada tanto por comillas simples ('cadena') como por comillas dobles ("cadena"), ambas identifican los caracteres alfanuméricos que constituyen un valor constante literal de tipo **str**.

El uso de comillas simples o dobles es intercambiable y depende de las preferencias del programador o de la necesidad de incluir comillas dentro de la cadena.

```
"Esta es una cadena de caracteres"
```

```
'Esta es una cadena de caracteres'
```

```
'Se pueden representar con comillas simples'
```

```
'Se pueden representar con comillas simples'
```

```
"Las cadenas se pueden " + "sumar"
```

```
'Las cadenas se pueden sumar'
```



En ocasiones, es necesario trabajar con cadenas de caracteres extensas distribuidas a lo largo de varias líneas en Python. Para lograr esto, simplemente se encierra la cadena entre triples comillas. Este enfoque permite la creación de cadenas multilínea de manera más legible y cómoda.

```
"""Esta es una cadena  
distribuida en  
3 líneas"""
```



Formateo de cadena de caracteres: En Python existen distintas formas de formatear cadenas de caracteres. En esta sección veremos algunas de ellas

El operador "%" se utiliza para formatear una cadena que contiene texto normal junto con "los especificadores de argumentos" y símbolos especiales como: "%s", "%d", "%f", "%.".

- %s - Cadena de caracteres (String o cualquier objeto como los números representados en forma de texto)
- %d - Números enteros (int)
- %f - Números de punto flotante (float)
- %. - Números de punto flotante con una cantidad de decimales fijos a la derecha del punto

```
Main.py  X
Main.py > ...
1  first_name = "Juanjo"
2  last_name = "Garcia"
3  age = 38
4  height = 1.85555
5
6  formatted_str = "Hola mi nombre es %s %s, tengo %d años y mido %.2f" %(first_name, last_name, age, height)
7  print(formatted_str)
```

Salida:

```
python-pruebas 16:11 python .\Main.py
Hola mi nombre es Juanjo Garcia, tengo 38 años y mido 1.86
```



Formateo de cadena de caracteres: A partir de la versión 3 de python también tenemos la siguiente forma de formatear cadenas de caracteres.

Para este ejemplo, usamos las llaves para definir donde va a ir el valor que luego se va a pasar a la función format. Podemos usar la sintaxis de `'{:.' y número de decimales, para formatear ese valor. Podemos observar un ejemplo en el siguiente código:`

```
first_name = "Juanjo"
last_name = "Garcia"
age = 38
height = 1.85555

formatted_str = 'Hola mi nombre es {} {}, tengo {} años y mido {:.2f}'.format(first_name, last_name, age, height)
print(formatted_str)
```

Salida:

```
• >> python-pruebas 16:11 python .\Main.py
  Hola mi nombre es Juanjo Garcia, tengo 38 años y mido 1.86
```



F-Strings: A partir de la versión 3.6 de python tenemos otra opción para el formateo de cadenas de caracteres que es la 'Interpolación'. Este método consiste en poner la letra 'f' delante de las comillas y dentro de las mismas, intercaladas dentro del texto, llaves con el nombre de la variable que se va a insertar en el texto. Con este método también podemos utilizar la sintaxis que se explica en la diapositiva anterior y formatear, por ejemplo, el valor de una variable de tipo float para limitar el número de decimales en la salida como podemos observar en el siguiente ejemplo de código:

```
first_name = "Juanjo"
last_name = "Garcia"
age = 38
height = 1.85555

formatted_str = f'Hola mi nombre es {first_name} {last_name}, tengo {age} años y mido {height:.2f}'
print(formatted_str)
```

Salida:

```
• >> python-pruebas 16:11 python .\Main.py
  Hola mi nombre es Juanjo Garcia, tengo 38 años y mido 1.86
```



F-Strings:

- Permiten la evaluación de expresiones directamente dentro de las llaves {}. Facilitan el formateo de números, cómo redondear decimales.
- Pueden incluir expresiones condicionales para mostrar diferentes resultados según una condición.
- En Python 3.8 y versiones posteriores, puedes añadir = al final de una expresión dentro de un f-string para imprimir tanto la expresión como su valor.

```
a = 5
b = 10
print(f"Cinco más diez es {a + b}.")
```

```
num = 87
print(f"¿Es num par? {True si num%2==0 else False}")
```

```
# Ejemplo usando Python 3.8 y versiones posteriores
nombre = "Alice"
edad = 30

# Imprime tanto el nombre de la variable como su valor
print(f"{nombre=}, {edad=}") # Imprimirá nombre='Alice', edad=30
```

```
# Ejemplo usando Python 3.7 y versiones anteriores
nombre = "Alice"
edad = 30

# Necesitas escribir la variable o expresión dos veces para obtener la misma
print(f"nombre={nombre}, edad={edad}") # Imprimirá nombre=Alice, edad=30
```



Acceso a los caracteres de un string: En Python, una cadena de texto (string) es una secuencia ordenada de caracteres. Esto significa que cada carácter en la cadena tiene una posición específica, también conocida como índice.

Los índices en Python comienzan desde 0, no desde 1.

La sintaxis para acceder a cada uno de esos caracteres es 'variable[posición]'.

```
name = "Juanjo Garcia"

first_letter = name[0]
second_letter = name[1]
third_letter = name[2]

print(f'1- {first_letter}, 2- {second_letter}, 3- {third_letter}')
```

Salida:

```
● >> python-pruebas 07:54 python .\Main.py
1- J, 2- u, 3- a
```



Acceso a los caracteres de un string: Otra forma de acceder a los caracteres de un String en python es el 'slicing'.

Este método extrae partes concretas de un String y consiste en que dentro de los corchetes indicamos porque caracter queremos comenzar a extraer y hasta que carácter queremos llegar (no incluido). Si el valor es negativo comenzamos por el final de la cadena, siendo el último carácter el de la posición -1.

```
name = "Juanjo Garcia"

first_name = name[:6]
last_name = name[7:]
last_letter = name[-1]
inverse_last_name = name[-6:]

print(f'{first_name}, {last_name}, {last_letter}, {inverse_last_name}')
```

Salida:

```
• >> python-pruebas 13:33 python .\Main.py
Juanjo, Garcia, a, Garcia
```



Caracteres de escape: Además de los caracteres convencionales que representan letras del alfabeto (mayúsculas y minúsculas), dígitos y otros signos diversos, se utiliza una codificación especial para referenciar caracteres especiales en Python. Estos caracteres especiales inician con el signo `\` (conocido como carácter de escape en este contexto) seguido de un código que lo identifica.

Carácter de escape	Descripción	Ejemplo	Salida Esperada
<code>\n</code>	Cambio de línea	"Hola\nMundo"	Hola Mundo
<code>\t</code>	Tabulador	"Nombre:\tJuan"	Nombre: Juan
<code>\\</code>	Barra invertida	"C:\\Usuario\\Directorio"	C:\\Usuario\\Directorio
<code>\'</code>	Comilla simple	'Texto con comillas simples: \'Hola\''	Texto con comillas simples: 'Hola'
<code>\''</code>	Comilla doble	"Texto con comillas dobles: \'Mundo\''"	Texto con comillas dobles: "Mundo"
<code>\b</code>	Retroceso (backspace)	"abc\bde"	abde
<code>\dd</code>	Valor octal ASCII (dos dígitos)	"Carácter con valor octal: \110\151"	Carácter con valor octal: Hi
<code>\xhh</code>	Valor hexadecimal ASCII (dos dígitos)	"Carácter con valor hexadecimal: \x48\x69"	Carácter con valor hexadecimal: Hi



Caracteres de escape: En las dos siguientes diapositivas se muestran algunos de los métodos más utilizados al operar con cadenas de caracteres en Python.

Método	Descripción	Ejemplo	Salida Esperada
len()	Devuelve la longitud de la cadena (número de caracteres).	<code>len("Hola")</code>	4
upper()	Convierte toda la cadena a mayúsculas.	<code>"Hola Mundo".upper()</code>	HOLA MUNDO
lower()	Convierte toda la cadena a minúsculas.	<code>"Hola Mundo".lower()</code>	hola mundo
replace(old, new)	Reemplaza todas las apariciones de la subcadena "old" con la subcadena "new".	<code>"Hola Mundo".replace("Hola", "Adiós")</code>	Adiós Mundo
strip()	Elimina espacios en blanco y caracteres de nueva línea al principio y al final de la cadena.	<code>" Hola Mundo ".strip()</code>	Hola Mundo
split(separator)	Divide la cadena en una lista de subcadenas utilizando el separador especificado.	<code>"Hola, Mundo".split(",")</code>	<code>['Hola', ' Mundo']</code>
join(iterable)	Concatena cada elemento de un iterable (lista, tupla, etc.) con la cadena como separador.	<code>" ".join(["Hola", "Mundo"])</code>	Hola Mundo
find(substring)	Devuelve el índice de la primera aparición de la subcadena o -1 si no se encuentra.	<code>"Hola Mundo".find("Mundo")</code>	5



Caracteres de escape:

Método	Descripción	Ejemplo	Salida Esperada
startswith(prefix)	Comprueba si la cadena comienza con el prefijo especificado.	"Hola Mundo".startswith("Hola")	True
endswith(suffix)	Comprueba si la cadena termina con el sufijo especificado.	"Hola Mundo".endswith("Mundo")	True
isdigit()	Verifica si todos los caracteres de la cadena son dígitos.	"12345".isdigit()	True
isalpha()	Verifica si todos los caracteres de la cadena son letras.	"Hola".isalpha()	True
isalnum()	Verifica si todos los caracteres de la cadena son alfanuméricos (letras o dígitos).	"Hola123".isalnum()	True
isspace()	Verifica si todos los caracteres de la cadena son espacios en blanco.	" ".isspace()	True
index(substring)	Devuelve el índice de la primera aparición de la subcadena o lanza un error si no se encuentra.	"Hola Mundo".index("Mundo")	5

Tipo de lista

6



Las listas en python son un tipo de colección que sirven para guardar colecciones de datos que pueden ser de distinto tipo.

Tienen las siguientes características:

- **Ordenada:** los elementos dentro de ella están indexados y se accede a ellos a través de su índice
- **Editable:** los elementos pueden editarse, añadir nuevos o eliminar los que ya tiene
- **Dinámica:** puede contener diferentes tipos de datos y objetos. Esto hace que puedan soportar paquetes multidimensionales de datos como por ejemplo un array, otra lista o muchos objetos
- **No única:** permite elementos duplicados



Las listas en Python se definen colocando el contenido entre corchetes, separando cada elemento por comas. Cada posición de la lista puede contener elementos de tipos diferentes y son mutables, lo que significa que sus elementos pueden ser modificados. En Python, los elementos de una lista se numeran desde 0 hasta longitud - 1. Se pueden aplicar numerosas funciones a una lista.

```
python  
  
mi_lista = [1, 2.0, "3", True, [1, 2]]
```

El concepto de lista impone una relación de orden, estableciendo una secuencia. Es importante notar que los índices de una lista comienzan desde 0 y van hasta longitud - 1.

Para crear una lista vacía, se puede utilizar una de las siguientes formas:

```
python  
  
lista_vacia = []  
# 0  
lista_vacia_otra_forma = list()
```

Ambas formas crean una lista sin elementos, que puede ser posteriormente modificada y ampliada.



Acceso en listas: Para acceder a un elemento específico de una lista en Python, se utiliza la notación de corchetes indicando la posición deseada. Si se proporciona un número negativo, se comenzará a recorrer la lista desde el final.

```
python

lista = [1, 2, 3, 4, 5]

# Acceder al primer elemento
elemento_1 = lista[0]

# Acceder al cuarto elemento
elemento_4 = lista[3]

# Acceder al tercer elemento desde el final
elemento_ultimo_3 = lista[-3]
```

Acceso con índices positivos

['banana', 'orange', 'mango', 'lemon']

0 1 2 3

Acceso con índices negativos

['banana', 'orange', 'mango', 'lemon']

-4 -3 -2 -1



Acceso en listas: Para acceder a un rango dentro de una lista, se utilizan diferentes opciones:

- Desde el inicio, tomar 'x' elementos
- Desde la posición 'x' (incluida), tomar todos los elementos hasta el final
- Desde la posición 'x' (incluida) hasta la posición 'y'
- Desde la posición 'x', tomar todos los elementos hasta el final iterando la lista de dos en dos

```
sublista_inicio = lista[:x]
```

```
sublista_desde_x = lista[x:]
```

```
sublista_entre_x_y = lista[x:y]
```

```
sublista_desde_x_de_dos_en_dos = lista[x::2]
```



Unpacking y slicing: El unpacking en Python nos permite asignar una lista a múltiples variables en una misma línea de código. Nos permite ‘desempaquetar’ los elementos en variables individuales o incluso una parte de esa lista en otra.

Para que la operación no arroje un error, el número de objetos separados por comas a la izquierda del signo igual debe coincidir con el número de elementos de la lista, e incluso con el ‘slicing’ podemos desempaquetar un número determinado de ellos en otra lista.

Unpacking:

```
lst = ['item1','item2','item3', 'item4', 'item5']
first_item, second_item, third_item, *rest = lst
print(first_item)      # item1
print(second_item)     # item2
print(third_item)      # item3
print(rest)            # ['item4', 'item5']
```

Slicing:

```
# Slicing con índices positivos
fruits = ['banana', 'orange', 'mango', 'lemon']
all_fruits = fruits[0:4] # Devuelve todas las frutas
all_fruits = fruits[0:] # Comienza con el índice 0 y hasta el final
orange_and_mango = fruits[1:3] # Comienza en el índice 1 hasta el 3 no incluido
orange_and_lemon = fruits[1::2] # Al utilizar dobles puntos comienza en el índice 1 y obtiene cada dos elementos ['orange', 'lemon']

# Slicing con índices negativos
fruits = ['banana', 'orange', 'mango', 'lemon']
all_fruits = fruits[-4:] # Devuelve todas las frutas
orange_and_mango = fruits[-3:-1] # No incluye el último elemento del índice (-1), ['orange', 'mango']
orange_mango_lemon = fruits[-3:] # Comienza en el índice -3 hasta el final, ['orange', 'mango', 'lemon']
reverse_fruits = fruits[::-1] # Devuelve la lista en orden inverso, ['lemon', 'mango', 'orange', 'banana']
```



Método	Descripción	Ejemplo	Salida Esperada
append(elemento)	Agrega un elemento al final de la lista.	lista = [1, 2, 3] lista.append(4)	[1, 2, 3, 4]
extend(iterable)	Agrega los elementos de un iterable al final de la lista.	lista1 = [1, 2, 3] lista2 = [4, 5] lista1.extend(lista2)	[1, 2, 3, 4, 5]
insert(pos, elemento)	Inserta un elemento en la posición especificada.	lista = [1, 2, 3] lista.insert(1, 4)	[1, 4, 2, 3]
remove(elemento)	Elimina la primera ocurrencia del elemento especificado.	lista = [1, 2, 3, 4, 2] lista.remove(2)	[1, 3, 4, 2]
pop([pos])	Elimina y devuelve el elemento en la posición especificada, o el último si no se proporciona pos.	lista = [1, 2, 3] elemento = lista.pop(1)	elemento será 2, lista será [1, 3]
index(elemento)	Devuelve el índice de la primera ocurrencia del elemento.	lista = [1, 2, 3] indice = lista.index(2)	indice será 1
sort()	Ordena los elementos de la lista de forma ascendente.	lista = [3, 1, 4, 1, 5, 9, 2] lista.sort()	[1, 1, 2, 3, 4, 5, 9]
reverse()	Invierte el orden de los elementos en la lista.	lista = [1, 2, 3] lista.reverse()	[3, 2, 1]

Tipo tupla

7



Las tuplas en Python son similares a las listas, pero en lugar de utilizar corchetes se definen mediante paréntesis. La principal característica de las tuplas es que son inmutables, es decir, una vez creadas, no pueden ser modificadas.

```
python  
  
mi_tupla = (1, 2, 3, "cuatro")
```

También se pueden crear tuplas de la siguiente forma:

```
mi_tupla = tuple((1, 2, 3, "cuatro"))
```

O por ejemplo a partir de una Lista:

```
mi_lista = [1, 2, 3, "cuatro"]  
mi_tupla = tuple(mi_lista)
```

La ventaja de las tuplas sobre las listas radica en su inmutabilidad, lo que las hace más rápidas y eficientes en cuanto a uso de memoria. A diferencia de las listas, no se pueden modificar durante la ejecución del programa.

Es posible crear tuplas de un solo elemento, aunque para evitar ambigüedades, se debe agregar una coma al final:

```
un_elemento = (10,)
```



Al igual que con las listas, se puede acceder a los elementos de las tuplas mediante el operador de indexación `[]`.

```
primer_elemento = mi_tupla[0]
```

Acceso con índices positivos

('banana', 'orange', 'mango', 'lemon')

0 1 2 3

Acceso con índices negativos

('banana', 'orange', 'mango', 'lemon')

-4 -3 -2 -1

Al igual que con las Listas, con las Tuplas tenemos el 'unpacking' que nos permite asignar una Tupla a múltiples variables en una misma línea de código. Nos permite 'desempaquetar' los elementos en variables individuales o incluso una parte de esa Tupla en otra y el 'slicing' con el cual podemos desempaquetar un número determinado de ellos en otra Tupla. (ver diapositiva en el apartado de Listas)



Método	Descripción	Ejemplo	Salida Esperada
len(elemento)	Devuelve el número de elementos en la tupla	tupla = (1, 2, 3) len(tupla)	3
max(tupla)	Devuelve el elemento máximo en la tupla	tupla = (1, 2, 3) max(tupla)	3
min(tupla)	Devuelve el elemento mínimo en la tupla	tupla = (1, 2, 3) min(tupla)	1
sum(tupla)	Devuelve la suma de los elementos en la tupla (deben ser numéricos)	tupla = (1, 2, 3) sum(tupla)	6
sorted(tupla)	Devuelve una lista con los elementos de la tupla ordenados	tupla= (2, 3, 1) sorted(tupla)	[1, 2, 3]
any(tupla)	Devuelve 'true' si al menos uno de los elementos de la tupla es verdadero	tupla= (0, 1, 2) any(tupla)	True
all(tupla)	Devuelve 'true' si todos los elementos de la tupla son verdaderos	tupla= (1, 2, 3) any(tupla)	True

Tipo diccionario

8



Los diccionarios en Python son colecciones iterables, no secuenciales y mutables de elementos compuestos por una clave única que identifica al elemento y el valor que se desea almacenar. Es importante destacar que no pueden existir elementos con la misma clave. La sintaxis de un diccionario es la siguiente:

```
python Copy code  
  
mi_dict = {clave_1: dato_1, clave_2: dato_2, ..., clave_n: dato_n}
```

Los valores pueden ser de cualquier tipo, incluyendo datos simples, listas e incluso otros diccionarios, con cualquier nivel de anidamiento. Sin embargo, las claves deben ser de tipos inmutables, como enteros, cadenas o tuplas.

Para acceder a los elementos de un diccionario, se utiliza la clave entre corchetes:

```
python  
  
valor = mi_dict['clave']
```

Para realizar un acceso seguro, evitando errores si la clave no está presente, se utiliza el método **get()**, que devuelve **None** en caso de que la clave no aparezca en el diccionario.

```
valor_seguro = mi_dict.get('clave')
```



Para añadir un nuevo elemento a un diccionario existente en Python, se utiliza el operador de asignación `=`. A la izquierda del operador, se coloca el objeto diccionario con la nueva clave entre corchetes `[]`, y a la derecha, se especifica el valor que se asocia a esa clave.

```
python  
  
mi_dict['tres'] = 3
```

Es importante destacar que, si la clave ya existe en el diccionario, su valor se actualiza con el nuevo valor proporcionado.

Adicionalmente, existe el método `setdefault(clave[, valor])`. Este método devuelve el valor asociado a la clave si esta ya existe en el diccionario. En caso contrario, asigna la clave con el valor proporcionado como segundo argumento. Si no se especifica este segundo argumento, por defecto es `None`.

```
python Copy code  
  
valor_existente = mi_dict.setdefault('tres', 300) # Devuelve 3 (valor  
valor_nuevo = mi_dict.setdefault('cuatro', 4) # Asigna 'cuatro': 4 y
```

Este método proporciona una manera conveniente de añadir claves con valores predeterminados a un diccionario si la clave aún no existe.



- ***pop(clave [, valor por defecto])***: Si la clave está presente en el diccionario, este método la elimina y devuelve su valor. Si la clave no está presente, devuelve el valor por defecto proporcionado. Si no se proporciona un valor por defecto y la clave no está en el diccionario, se lanza la excepción *KeyError*.

```
valor_eliminado = mi_dict.pop('tres', None) # Elimina 'tres'
```

- ***popitem()***: Elimina el último par clave-valor del diccionario y lo devuelve. Si el diccionario está vacío, se lanza la excepción *KeyError*. Es importante mencionar que en versiones anteriores a Python 3.7, este método eliminaba y devolvía un par clave-valor aleatorio, sin garantizar que fuera el último.

```
ultimo_par = mi_dict.popitem() # Elimina y devuelve el último par clave-valor
```

- ***del d[clave]***: Utilizando la instrucción `del`, se elimina el par clave-valor correspondiente a la clave proporcionada. Si la clave no existe, se lanza la excepción *KeyError*.

```
del mi_dict['tres'] # Elimina el par clave-valor con clave 'tres'
```

- ***clear()***: Borra todos los pares clave-valor del diccionario, dejándolo vacío.

```
mi_dict.clear() # Borra todos los pares clave-valor del diccionario
```



Adicionalmente a lo ya explicado se muestran a continuación más métodos usualmente utilizados en diccionarios

Método	Descripción	Ejemplo	Salida Esperada
keys()	Devuelve una vista de las claves del diccionario.	diccionario = {'a': 1, 'b': 2} claves = diccionario.keys()	['a', 'b']
values()	Devuelve una vista de los valores del diccionario.	diccionario = {'a': 1, 'b': 2} valores = diccionario.values()	[1, 2]
items()	Devuelve una vista de pares (clave, valor) del diccionario.	diccionario = {'a': 1, 'b': 2} items = diccionario.items()	[('a', 1), ('b', 2)]

Tipo set

9



Los conjuntos (sets) en Python son colecciones no ordenadas de objetos únicos. Tienen las siguientes características:

1. La colección es mutable, pero los elementos contenidos deben ser inmutables.
2. Los elementos no pueden aparecer repetidos.
3. Es iterable y no secuencial.

Para crear conjuntos, se pueden utilizar llaves {} de la siguiente manera:

```
mi_conjunto = {dato_1, dato_2, ..., dato_n}
```

También es posible crear conjuntos utilizando la función `set()`, a la que se le pasa como argumento algún otro iterable:

```
mi_otro_conjunto = set((dato_1, dato_2, ..., dato_n))
```

En ambos casos, los conjuntos proporcionan una forma eficiente de almacenar elementos únicos y realizar operaciones de conjunto, como la unión, la intersección y la diferencia. Los elementos en un conjunto deben ser inmutables, pero el conjunto en sí es mutable y puede ser modificado después de su creación.



Método	Descripción	Ejemplo	Salida Esperada
add(elemento)	Agrega un elemento al conjunto.	st = {1, 2, 3} st.add(4)	{1, 2, 3, 4}
remove(elemento)	Elimina el elemento del conjunto.	st = {1, 2, 3} st.remove(2)	{1, 3}
discard(elemento)	Elimina el elemento del conjunto si está presente.	st = {1, 2, 3} st.discard(2)	{1, 3}
pop()	Elimina y devuelve un elemento aleatorio del conjunto.	st = {1, 2, 3} st1 = st.pop()	st = {1, 2} st1 = 3
clear()	Elimina todos los elementos del conjunto.	st = {1, 2, 3} st.clear()	{}
union(set)	Devuelve un nuevo set con los valores de dos sets	st1 = {1, 2, 3} st2 = {4, 5, 6} st1.union(st2)	{1, 2, 3, 4, 5, 6}



Método	Descripción	Ejemplo	Salida Esperada
update(set)	Inserta los elemento de un set en otro dado.	st1 = {1, 2, 3} st2 = {4, 5, 6} st1.update(st2)	st1 = {1, 2, 3, 4, 5, 6}
intersection(set)	Devuelve los elementos comunes en ambos sets.	st1 = {1, 2, 3, 4, 5, 6} st2 = {3, 2, 8} st3 = st1.intersection(st2)	st3 = {3, 2}
issubset(set)	Devuelve true si es el set pasado es subset	st1 = {1, 2, 3, 4, 5, 6} st2 = {3, 2} st2.issubset(st1)	True
issuperset(set)	Devuelve true si es el set pasado es superset	st1 = {1, 2, 3, 4, 5, 6} st2 = {3, 2} st1.issuperset(st2)	True
difference(set)	Devuelve la diferencia entre dos sets	st1 = {1, 2, 3, 4, 5} st2 = {3, 2} st1.difference(st2)	{1, 4, 5}

Colecciones

10



El módulo *collections* en Python proporciona nuevas estructuras de datos mejoradas, alternativas a las estructuras de datos básicas integradas. Ofrece herramientas especializadas para abordar diversas necesidades de programación como, por ejemplo:

- Clase ***Counter*** para contar elementos en una secuencia
- Clase ***defaultdict*** para crear diccionarios con valores predeterminados
- Clase ***namedtuple*** para crear tuplas con campos nombrados, mejorando la claridad del código
- Clase ***deque*** la cual ofrece operaciones eficientes de doble extremo en colas
- Clase ***OrderedDict*** que mantiene el orden de inserción en los diccionarios

Estas herramientas enriquecen la funcionalidad estándar de Python, facilitando la implementación de soluciones más eficientes y legibles para una variedad de problemas de programación.

Además de las mencionadas, existen algunas clases más ofrecidas por este módulo: *UserDict*, *UserList* y *UserString*. Actúan a modo de *wrapper* (clase contenedora) de las correspondientes estructuras de datos básicas (*dict*, *list* y *string*). Se utilizan cuando se pretende crear una clase propia con funcionalidades nuevas o modificadas respecto a las estructuras básicas.

Fechas

11



El módulo ***datetime*** en Python proporciona clases para manipular fechas y horas de manera eficiente. Este módulo facilita la creación, manipulación y formateo de objetos de fechas y horas, así como la realización de cálculos temporales. Algunas de sus clases más usadas son:

- Clase ***date***: Representa una fecha (año, mes, día).
- Clase ***time***: Maneja solo la parte de hora (horas, minutos, segundos).
- Clase ***datetime***: Combina fecha y hora en un solo objeto.

El módulo ***datetime*** también cuenta con otras clases que nos pueden ser de utilidad:

- Clase ***timedelta***: Representa la diferencia de tiempo entre dos objetos ***date*** o entre dos objetos ***datetime***.
- Clase ***tzinfo***: Permite trabajar con zonas horarias.

Estas herramientas permiten realizar operaciones como calcular la diferencia entre fechas, sumar o restar días, horas o minutos, y trabajar con zonas horarias globales.

Además de estas clases, el módulo ***datetime*** incluye funcionalidades adicionales para formatear y analizar fechas y horas mediante los métodos ***strftime()*** (formateo a cadenas) y ***strptime()*** (conversión de cadenas a objetos de fecha).

Para más información sobre el módulo ***datetime*** puedes utilizar consultar la [documentación oficial](#).



Algunos de los métodos más útiles y comunes de la clase *date* son los siguientes:

Método	Descripción	Ejemplo	Salida Esperada
today()	Obtiene la fecha actual.	<code>date.today()</code>	2024-09-09
fromisoformat(str)	Crea un objeto date desde una cadena en formato ISO.	<code>date.fromisoformat('2024-09-09')</code>	2024-09-09
isoformat()	Convierte un objeto date en una cadena en formato ISO.	<code>d = date(2024, 9, 9)</code> <code>d.isoformat()</code>	'2024-09-09'
replace()	Crea un nuevo objeto date reemplazando algunos de sus atributos.	<code>d = date(2024, 9, 9)</code> <code>new_d = d.replace(year=2023)</code>	2023-09-09
weekday()	Devuelve el día de la semana como un número (0=Lunes, 6=Domingo).	<code>d = date(2024, 9, 9)</code> <code>d.weekday()</code>	0



Algunos de los métodos más útiles y comunes de la clase *time* son los siguientes:

Método	Descripción	Ejemplo	Salida Esperada
isoformat()	Devuelve una cadena con la hora en formato ISO.	<pre>t = time(14, 30, 45) t.isoformat()</pre>	'14:30:45'
replace()	Crea un nuevo objeto time, modificando los valores de otro objeto time.	<pre>t = time(14, 30, 45) new_t = t.replace(hour=16)</pre>	16:30:45
strftime(str)	Formatea un objeto time a una cadena con el formato especificado.	<pre>t = time(14, 30, 45) t.strftime("%H:%M:%S")</pre>	'14:30:45'



Algunos de los métodos más útiles y comunes de la clase *datetime* son los siguientes:

Método	Descripción	Ejemplo	Salida Esperada
<code>now()</code>	Obtiene la fecha y hora actual.	<code>datetime.now()</code>	2024-09-09 14:30:45.123456
<code>combine()</code>	Combina un objeto date y un objeto time para crear un datetime.	<code>d = date(2024, 9, 9)</code> <code>t = time(14, 30)</code> <code>dt = datetime.combine(d, t)</code>	2024-09-09 14:30:00
<code>strptime()</code>	Convierte una cadena en un objeto datetime en base a un formato dado.	<code>datetime.strptime('09/09/2024 14:30', '%d/%m/%Y %H:%M')</code>	2024-09-09 14:30:00
<code>strftime()</code>	Formatea un datetime en una cadena.	<code>dt = datetime(2024, 9, 9, 14, 30)</code> <code>dt.strftime('%Y-%m-%d %H:%M')</code>	'2024-09-09 14:30'
<code>replace()</code>	Crea un nuevo objeto datetime modificando los valores de otro objeto datetime.	<code>dt = datetime(2024, 9, 9, 14, 30)</code> <code>new_dt = dt.replace(year=2023)</code>	2023-09-09 14:30:00

¡Gracias!